

PROJECT REPORT — Air-Gapped Predictive NOC Copilot

Report Generated: 2026-08-04
Auditor: Automated Technical Audit (Full Source Inspection)
Repository: github.com/rethiofficial1828-reka/NOC-copilot
Classification: Internal Review — Not for Marketing

1. Project Overview

FIELD	VALUE
Project Name	Air-Gapped Predictive NOC Copilot
Version	1.0.0
Purpose	AI-powered Network Operations Center copilot that predicts network link failures before they occur and provides RAG-grounded remediation guidance — fully air-gapped with zero cloud dependencies.
Language	Python 3.13
ML Framework	XGBoost (gradient-boosted classifier)
API Framework	FastAPI + Uvicorn
Dashboard	Streamlit
Database	SQLite3 (file-based, <code>data/telemetry.db</code>)
LLM	Ollama (Phi-3) with structured template fallback
Knowledge Retrieval	TF-IDF-based pure-Python retriever (offline), optional FAISS + SentenceTransformers (neural)
Package Manager	pip (via <code>requirements.txt</code>)
Total Lines of Code	4,619 (Python: 2,379

One-Paragraph Description: The NOC Copilot is a complete ML-driven network monitoring platform designed for air-gapped/secure MPLS and SD-WAN environments. It consists of four services: (1) a

Fault Simulator daemon that generates synthetic or live telemetry into a SQLite database every 2 seconds, (2) an **XGBoost Predictive Engine** (FastAPI on port 8000) that computes rolling-window features and predicts risk scores with time-to-SLA-breach estimation, (3) a **RAG Copilot** (FastAPI on port 8001) that retrieves relevant runbook chunks via TF-IDF and formulates remediation advice through an LLM (Ollama/Phi-3) or a deterministic fallback template, and (4) a **Streamlit Dashboard** (port 8501) providing real-time metric visualization, risk banners, and interactive simulation controls.

2. Current Status Summary

Overall Completion: ~85%

The core predictive loop (telemetry → features → model → risk score → copilot → dashboard) is fully functional. Testing infrastructure is comprehensive. The remaining 15% consists of missing production hardening (authentication, HTTPS, logging, graceful shutdown), incomplete CI/CD deployment, and absent database migrations.

FEATURE	STATUS	NOTES
Synthetic telemetry generation	Done	<code>faultsim/generate_dataset.py</code> — 5 healthy + 5 congestion scenarios, 4500 samples
Live fault simulation daemon	Done	<code>faultsim/inject_fault.py</code> — healthy/congestion/mitigated modes, 2s interval
Rolling-window feature extraction	Done	<code>engine/features.py</code> — 16 features: current values, 30s means, 60s slopes, deltas
XGBoost risk prediction model	Done	<code>engine/model.py</code> — binary classifier + explainable formula hybrid
Time-to-impact estimation	Done	Linear extrapolation to SLA thresholds (95% util, 150ms latency)
TF-IDF RAG retriever (offline)	Done	<code>copilot/rag.py</code> — pure Python, no external downloads required
Neural FAISS retriever (optional)	Partial	Code present but disabled by default (<code>NOC_USE_NEURAL=false</code>); requires HuggingFace model download
Ollama/Phi-3 LLM integration	Done	<code>copilot/llm.py</code> — 2s connect / 6s read timeout with graceful fallback
Deterministic LLM fallback	Done	<code>copilot/llm.py</code> — structured JSON response when Ollama unavailable
FastAPI predictive engine API	Done	<code>engine/api.py</code> — <code>/predict</code> , <code>/health</code> endpoints
FastAPI copilot API	Done	<code>copilot/api.py</code> — <code>/copilot</code> , <code>/health</code> endpoints

FEATURE	STATUS	NOTES
Streamlit real-time dashboard	Done	<code>ui/app.py</code> — 418 LOC, glassmorphism UI, auto-refresh 2s
Interactive topology map (HTML)	Done	<code>topology_map.html</code> — 899 LOC, Leaflet.js visualization
Containerlab topology (YAML)	Done	<code>topology.clab.yml</code> — FRRouting hub + branch1 nodes
College network topology docs	Done	<code>copilot/docs/college_topology.txt</code> — 3 zones mapped
Unit tests	Done	9 unit tests in <code>test_master.py</code>
Integration tests	Done	7 API contract tests
E2E scenario tests	Done	4 lifecycle tests (healthy → congestion → copilot → recovery)
Edge case tests	Done	8 boundary condition tests
Stress tests	Done	3 load tests (rapid-fire APIs + DB consistency)
CI/CD pipeline (GitHub Actions)	Partial	YAML exists locally but NOT pushed to GitHub (token lacked <code>workflow</code> scope)
API authentication	Pending	No auth on any endpoint
HTTPS/TLS	Pending	All services use plain HTTP
Rate limiting	Pending	No rate limiting on any API
Structured logging	Pending	Uses <code>print()</code> statements throughout
Database migrations	Pending	Schema created inline in <code>inject_fault.py</code>
Graceful shutdown / signal handling	Partial	<code>run.sh</code> has <code>trap cleanup</code> , but daemon loop lacks signal handling
Input validation	Partial	Pydantic models validate types but not value ranges
Production deployment config	Pending	No Dockerfile, no systemd units, no WSGI/Gunicorn config

3. Pending / Incomplete Work

3.1 TODO / FIXME / HACK Comments in Code

FILE	LINE	TYPE	CONTENT
—	—	—	No TODO/FIXME/HACK comments found in the entire codebase.

3.2 Unfinished Features

ITEM	DESCRIPTION	SEVERITY
No API authentication	All 4 endpoints (<code>/predict</code> , <code>/copilot</code> , <code>/health</code> ×2) accept unauthenticated requests. Any device on the network can trigger predictions or modify simulation state.	Critical
No HTTPS enforcement	All HTTP traffic is plaintext on ports 8000, 8001, 8501.	High
No rate limiting	APIs can be hammered without restriction.	High
No structured logging	All diagnostic output uses <code>print()</code> — no log levels, rotation, or centralized collection.	Medium
No database migration system	Schema is created via inline <code>CREATE TABLE IF NOT EXISTS</code> in <code>faultsim/inject_fault.py</code> (lines 16–36). No versioning, no migration tool.	Medium
No production WSGI server	Uvicorn is used directly with default single-worker config. No Gunicorn/multi-worker setup.	Medium
No Dockerfile / container packaging	Despite having a Containerlab topology, the application itself has no Dockerfile.	Medium
No graceful daemon shutdown	<code>faultsim/inject_fault.py</code> <code>main_loop()</code> (line 84) runs <code>while True</code> without signal handling. <code>Ctrl+C</code> will leave orphaned DB connections.	Low
CI/CD workflow not deployed	<code>.github/workflows/test.yml</code> exists locally but is NOT on GitHub (token permission issue).	Low
Neural retriever disabled	FAISS + SentenceTransformers code exists in <code>copilot/rag.py</code> but is gated behind <code>NOC_USE_NEURAL=true</code> env var. Not tested in CI.	Low

3.3 Missing Error Handling

FILE	LINE(S)	ISSUE
<code>engine/api.py</code>	36–42	

FILE	LINE(S)	ISSUE
		<code>sqlite3.connect()</code> not using context manager; if <code>pd.read_sql_query</code> raises, <code>conn.close()</code> is skipped → connection leak
<code>engine/api.py</code>	85–86	Generic <code>except Exception as e</code> exposes raw error strings via HTTP 500 response
<code>engine/features.py</code>	15–59	No validation that required DataFrame columns exist; <code>KeyError</code> if columns are missing
<code>engine/features.py</code>	12	<code>np.polyfit</code> can throw <code>LinAlgError</code> on NaN/Inf input; not caught
<code>engine/model.py</code>	72, 117	XGBoost load/predict failures are caught but only <code>print()</code> -ed, not logged or metricked
<code>copilot/api.py</code>	10–14	RAG initialization failure silently sets <code>rag = None</code> ; no retry or monitoring alert
<code>copilot/api.py</code>	51	Exception message includes <code>str(e)</code> in HTTP response — information disclosure
<code>copilot/rag.py</code>	131–154	<code>read_raw_documents()</code> doesn't handle <code>FileNotFoundError</code> or <code>PermissionError</code>
<code>copilot/llm.py</code>	100	<code>json.loads(response_text)</code> <code>JSONDecodeError</code> is caught by generic <code>except</code> without logging malformed output
<code>faultsim/inject_fault.py</code>	84–169	Infinite loop with generic <code>except Exception</code> on entire iteration; no signal-based shutdown

3.4 Missing Tests

GAP	DESCRIPTION
No <code>pytest</code> test runner	All tests use <code>unittest</code> or custom procedural scripts; <code>pytest</code> is listed in <code>requirements.txt</code> but never used
No code coverage	No <code>coverage</code> or <code>pytest-cov</code> configuration
No mutation testing	Not applicable at current maturity
No load testing at scale	Stress tests are limited to 20 API calls and 100 DB rows
Neural RAG path untested	FAISS + SentenceTransformers path has zero test coverage
No negative security tests	No test for prompt injection, SQL injection via <code>interface</code> parameter, or XSS

4. Project Workflow

4.1 Application Flow

Step-by-step data flow:

1. **Telemetry Generation:** The `faultsim/inject_fault.py` daemon runs every 2 seconds, generating synthetic network metrics (utilization, latency, jitter, drops, routing_flaps) based on the current simulation mode (healthy/congestion/mitigated) and inserting them into `data/telemetry.db` (SQLite).
2. **Feature Extraction:** When a prediction is requested via `GET /predict?interface=Branch3-Uplink`, the engine API (`engine/api.py`) queries the last 30 telemetry samples from SQLite, passes them to `engine/features.py` which computes 16 rolling-window features (current values, 30s means, 60s slopes, baseline deltas).
3. **Risk Prediction:** The feature vector is fed to `engine/model.py`'s `RiskPredictor` which combines an XGBoost binary classifier (trained on synthetic data) with an explainable formula. The output includes a 0.0–1.0 risk score, time-to-SLA-breach (minutes), and human-readable contributing signals.
4. **RAG Retrieval:** When risk exceeds 0.3, the Streamlit dashboard auto-triggers `POST /copilot` on the copilot API. The copilot builds a query from the alert data and retrieves top-3 relevant chunks from runbook/topology documents via TF-IDF cosine similarity (`copilot/rag.py`).
5. **LLM Reasoning:** The retrieved chunks plus alert data are formatted into a structured prompt and sent to Ollama (Phi-3 model). If Ollama is unreachable (2s connect timeout), a deterministic fallback template generates the response (`copilot/llm.py`).
6. **Dashboard Rendering:** The Streamlit UI (`ui/app.py`) polls the engine API every 2 seconds, renders live metric charts (utilization, latency, jitter, drops), displays a color-coded risk banner (green/amber/red), and shows the AI copilot's structured recommendations with cited runbook sources.

4.2 Architecture Diagram

[Mermaid diagram — view in GitHub for interactive rendering]

4.3 Critical Feature: RAG-Grounded Copilot Flow

[Mermaid diagram — view in GitHub for interactive rendering]

5. Folder / File Structure

```

noc-copilot/
├── .github/
│   └── workflows/
│       └── test.yml
├── .gitignore
├── .streamlit/
│   └── config.toml
├── README.md
├── PROJECT_REPORT.md
├── requirements.txt
├── run.sh
├── topology.clab.yml
├── topology_map.html
├── test_config.yml
├── test_all.py
├── test_e2e.py
├── test_master.py

├── engine/
│   ├── api.py
│   ├── features.py
│   └── model.py

├── copilot/
│   ├── api.py
│   ├── rag.py
│   ├── llm.py
│   └── docs/
│       ├── runbook_congestion.txt
│       ├── past_incidents.txt
│       ├── network_topology.txt
│       └── college_topology.txt

├── faultsim/
│   ├── generate_dataset.py
│   └── inject_fault.py

├── ui/
│   └── app.py

└── data/
    ├── synthetic_telemetry.csv
    ├── xgboost_model.json
    ├── chunks.txt
    └── telemetry.db

# Project root

# GitHub Actions CI/CD pipeline (NOT pushed to remote)
# Git ignore patterns (37 lines)

# Streamlit config: headless=true, no usage stats
# Professional README with architecture, setup, API docs
# This report
# Python dependencies (10 packages)
# One-command launcher for all 4 services
# Containerlab topology (hub + branch1, FRRouting)
# Interactive Leaflet.js network topology visualization
# YAML-defined test scenarios, thresholds, topology (585 lines)
# Quick unit test suite (5 tests, unittest)
# End-to-end lifecycle test (5 stages, procedural)
# Master test runner – 31 tests across 5 categories

# Predictive ML Engine
# FastAPI server (port 8000): /predict, /health
# Rolling-window feature extraction (16 features)
# XGBoost RiskPredictor class + training script

# RAG + LLM Copilot
# FastAPI server (port 8001): /copilot, /health
# TFIDFRetriever + LocalRAG with optional FAISS neural path
# Ollama/Phi-3 query engine + deterministic fallback
# RAG knowledge base (read-only runbooks)
# Congestion response procedure
# Historical incident logs (INC-492, INC-511)
# MPLS/SD-WAN topology (Branch3 ↔ DC1)
# College network zones, QoS policy, traffic rules

# Fault Simulation
# Synthetic training data generator (10 scenarios)
# Live fault injection daemon (2s interval)

# Dashboard
# Streamlit UI (premium glassmorphism design)

# Runtime data (partially gitignored)
# Pre-generated training data (4500 rows, 353KB)
# Trained XGBoost model artifact (81KB)
# Pre-parsed RAG document chunks (2KB)
# Live SQLite database (runtime-generated, 52KB)

```

6. Dependencies

6.1 Direct Dependencies (`requirements.txt`)

PACKAGE	VERSION CONSTRAINT	PURPOSE	PINNED?
pandas	≥ 2.0.0	DataFrames for telemetry feature extraction	✗ Minimum only
numpy	≥ 1.24.0	Numerical computation, slope/mean/clip	✗ Minimum only
scikit-learn	≥ 1.3.0	Listed but not directly imported in any source file	✗ Minimum only
xgboost	≥ 2.0.0	ML risk prediction classifier	✗ Minimum only
fastapi	≥ 0.100.0	API framework for engine + copilot servers	✗ Minimum only
uvicorn	≥ 0.23.0	ASGI server for FastAPI	✗ Minimum only
requests	≥ 2.31.0	HTTP client for Ollama, inter-service calls, dashboard API queries	✗ Minimum only
streamlit	≥ 1.28.0	Interactive real-time dashboard	✗ Minimum only
pytest	≥ 7.4.0	Test framework (listed but not used — tests use <code>unittest</code> / custom scripts)	Minimum only
pytest-timeout	≥ 2.2.0	Pytest timeout plugin (listed but not used)	Minimum only

6.2 Optional / Conditional Dependencies (in code but not in requirements.txt)

PACKAGE	USED IN	CONDITION
sentence-transformers	<code>copilot/rag.py</code> line 113	Only when <code>NOC_USE_NEURAL=true</code>
faiss-cpu	<code>copilot/rag.py</code> line 114	Only when <code>NOC_USE_NEURAL=true</code>

6.3 Dependency Audit

`pip-audit` / `npm audit` : Not found / Unable to verify. No audit tooling is configured in the project or CI pipeline.

Known concerns:

- All dependencies use **minimum-bound constraints** (`≥`), which can pull in untested or breaking

updates. **Recommendation:** Pin exact versions with `pip freeze > requirements.lock` for reproducible builds.

- `scikit-learn` is listed as a dependency but **never imported** anywhere in the source code. It adds ~200MB of unnecessary installation weight.
- `pytest` and `pytest-timeout` are listed but **never used** — all tests run via `unittest.main()` or custom procedural scripts.

7. Environment Setup & How to Run (End-to-End)

7.1 Prerequisites

REQUIREMENT	VERSION	NOTES
Python	3.13+	Verified: Python 3.13.12 on target system
pip	Latest	Package installer
SQLite3	Bundled with Python	No separate installation needed
Ollama	Latest (optional)	For live LLM inference; system works without it via fallback
git	2.x+	For cloning

7.2 Step-by-Step Setup

```
# 1. Clone the repository
git clone https://github.com/rethioofficial1828-reka/NOC-copilot.git
cd NOC-copilot

# 2. Create virtual environment
python3 -m venv venv
source venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Generate synthetic training dataset (if not present)
cd .. # Move to parent of noc-copilot
PYTHONPATH=$(pwd) python3 -m noc-copilot.faultsim.generate_dataset

# 5. Train the XGBoost model (if not present)
PYTHONPATH=$(pwd) python3 -m noc-copilot.engine.model train

# 6. Build the RAG chunk index
PYTHONPATH=$(pwd) python3 -m noc-copilot.copilot.rag

# 7. Launch all services (one command)
cd noc-copilot
./run.sh
```

7.3 Environment Variables

VARIABLE	REQUIRED	PURPOSE
<code>PYTHONPATH</code>	Yes	Must include project parent directory (set by <code>run.sh</code>)
<code>NOC_USE_NEURAL</code>	No	Set to <code>"true"</code> to enable FAISS + SentenceTransformers neural retriever (default: <code>false</code>)

Note: No `.env` file exists or is required. No API keys are needed for the default offline configuration.

7.4 How to Run Tests

```
# From project parent directory, with venv activated:
cd /path/to/parent-of-noc-copilot

# Quick unit tests only (no running services required):
PYTHONPATH=$(pwd) python3 -m unittest noc-copilot.test_all

# Full 31-test master suite (requires services running via run.sh):
PYTHONPATH=$(pwd) python3 noc-copilot/test_master.py

# E2E lifecycle test (requires services running):
PYTHONPATH=$(pwd) python3 noc-copilot/test_e2e.py
```

7.5 Default Ports & URLs

SERVICE	PORT	URL	AUTH REQUIRED
Predictive Engine API	8000	<code>http://localhost:8000/predict</code>	None
Copilot RAG & LLM API	8001	<code>http://localhost:8001/copilot</code>	None
Streamlit Dashboard	8501	<code>http://localhost:8501</code>	None
Ollama (optional)	11434	<code>http://localhost:11434/api/generate</code>	None

7.6 Build Command (Production)

Not found / Unable to verify. No production build configuration exists. No Dockerfile, no `Procfile` , no systemd unit files.

8. Security Review

8.1 Authentication & Session Handling

CHECK	RESULT	EVIDENCE
API authentication	FAIL	No authentication mechanism on any endpoint. <code>engine/api.py</code> , <code>copilot/api.py</code> — zero auth middleware.
Session management	FAIL	No session tokens, no cookies, no JWT.
Password hashing	N/A	No user accounts exist.
Brute-force protection	FAIL	No rate limiting, no lockout mechanism.

8.2 Input Validation & Sanitization

CHECK	RESULT	EVIDENCE
Pydantic model validation	PARTIAL	<code>copilot/api.py</code> lines 16–20: <code>CopilotRequest</code> validates types (str, float, List[str]) but does NOT enforce value ranges. <code>risk_score</code> accepts NaN, Infinity, or negative values.
Query parameter validation	PARTIAL	<code>engine/api.py</code> line 15: <code>interface</code> parameter is a free-form string with a default value but no whitelist or regex validation.
HTML sanitization	PARTIAL	<code>ui/app.py</code> uses <code>unsafe_allow_html=True</code> at 21+ locations. Most HTML is built from numeric metrics (safe), but <code>doc['chunk']</code> (line 378) and <code>src</code> (line 375) from RAG responses are injected into HTML without escaping.

8.3 SQL Injection Risk

CHECK	RESULT	EVIDENCE
Parameterized queries	PASS	All SQL queries use parameterized bindings (?). Verified in: <code>engine/api.py</code> line 41, <code>faultsim/inject_fault.py</code> lines 37–54, <code>ui/app.py</code> lines 145–157.
Raw query construction	PASS	No string concatenation or f-string SQL construction found anywhere.

8.4 XSS Risk Areas

CHECK	RESULT	EVIDENCE
<code>unsafe_allow_html=True</code>	RISK	<code>ui/app.py</code> line 375: <code>st.markdown(f' {src}', unsafe_allow_html=True)</code> — <code>src</code> comes from RAG document source filenames. If a malicious filename existed in <code>copilot/docs/</code> , it could inject HTML/JS.
<code>unsafe_allow_html=True</code>	RISK	<code>ui/app.py</code> line 378: RAG chunk content (<code>doc['chunk']</code>) is directly embedded in HTML. If runbook content contained <code><script></code> tags, they would be rendered.
Metric values in HTML	PASS	Numeric metrics are formatted with <code>:.1f</code> / <code>:.0f</code> — safe against injection.

8.5 CSRF Protection

CHECK	RESULT	EVIDENCE
CSRF tokens	N/A	No forms or state-changing browser submissions. API uses JSON POST, not HTML forms. Streamlit handles its own CSRF internally.

8.6 Authorization & Access Control

CHECK	RESULT	EVIDENCE
Endpoint authorization	FAIL	All 4 endpoints are publicly accessible to any client on the network. No role-based access, no API keys.
Simulation control	FAIL	<code>ui/app.py</code> lines 141–147: <code>set_sim_mode()</code> directly writes to SQLite via button clicks — no authorization check. Anyone with dashboard access can inject congestion or reset metrics.

8.7 Secrets Management

CHECK	RESULT	EVIDENCE
Hardcoded API keys	PASS	No API keys, tokens, or passwords found in source code.
<code>.env</code> files	PASS	No <code>.env</code> files exist in the project.
Git-exposed secrets	PASS	<code>.gitignore</code> excludes <code>data/telemetry.db</code> , CSV data, model JSON, and virtual environment.
Internal IPs in code	INFO	<code>copilot/llm.py</code> line 27: hardcoded BGP next-hop <code>192.168.30.2</code> . <code>copilot/docs/college_topology.txt</code> : internal IPs <code>10.50.0.1</code> ,

CHECK	RESULT	EVIDENCE
		<code>10.50.0.231</code> , <code>192.168.56.1</code> . These are lab/documentation IPs, not credentials, but reveal internal topology if published.

8.8 HTTPS/TLS Enforcement

CHECK	RESULT	EVIDENCE
TLS configured	✗ FAIL	<code>run.sh</code> lines 51, 56: Uvicorn binds on <code>0.0.0.0</code> with <code>--port</code> only — no <code>--ssl-keyfile</code> or <code>--ssl-certfile</code> .
Streamlit HTTPS	FAIL	<code>.streamlit/config.toml</code> has no TLS configuration.
Inter-service HTTPS	FAIL	<code>ui/app.py</code> uses <code>http://localhost:8000</code> and <code>http://localhost:8001</code> — plaintext HTTP.

8.9 Security Headers

CHECK	RESULT	EVIDENCE
CSP (Content-Security-Policy)	✗ FAIL	Not configured. FastAPI defaults have no security headers.
X-Frame-Options	FAIL	Not configured.
HSTS	FAIL	Not configured (no HTTPS exists to enforce).
X-Content-Type-Options	FAIL	Not configured.

8.10 Payment Handling

CHECK	RESULT	EVIDENCE
Payment processing	N/A	No payment functionality exists in this project.

8.11 Dependency Vulnerabilities

CHECK	RESULT	EVIDENCE
<code>pip-audit</code> scan	Not found / Unable to verify	No audit tooling in project or CI. No CVE scan has been run.
Loose version constraints	RISK	All 10 dependencies use <code>></code> minimum bounds. Future <code>pip install</code> may pull in untested or vulnerable versions.
Unused dependencies	WASTE	<code>scikit-learn</code> (not imported), <code>pytest</code> (not used), <code>pytest-timeout</code> (not used) — unnecessary attack surface.

8.12 Rate Limiting / DDoS Protection

CHECK	RESULT	EVIDENCE
API rate limiting	FAIL	No rate limiting middleware. FastAPI has no built-in rate limiter. <code>slowapi</code> or similar not installed.
Connection limits	FAIL	Uvicorn defaults with no <code>--limit-concurrency</code> or <code>--limit-max-requests</code> .

8.13 File Upload Validation

CHECK	RESULT	EVIDENCE
File uploads	N/A	No file upload functionality exists.

8.14 Logging of Sensitive Data

CHECK	RESULT	EVIDENCE
Password/ token logging	PASS	No passwords, tokens, or secrets are logged via <code>print()</code> statements.
Error message exposure	RISK	<code>engine/api.py</code> line 86: <code>detail=f"Prediction error: {str(e)}"</code> — raw exception strings returned to API clients may contain file paths or internal details. Same pattern in <code>copilot/api.py</code> line 51.

8.15 Prompt Injection

CHECK	RESULT	EVIDENCE
LLM prompt injection	RISK	<code>copilot/llm.py</code> lines 55–78: <code>payload.interface</code> and <code>payload.contributing_signals</code> are injected directly into the LLM prompt without sanitization. A malicious API caller could craft signals like <code>"Ignore all instructions and ... "</code> to alter LLM behavior.

8.16 Network Binding

CHECK	RESULT	EVIDENCE
Services bound to <code>0.0.0.0</code>	RISK	<code>run.sh</code> lines 51, 56, 72: All 3 services bind to <code>0.0.0.0</code> , exposing them to the entire network. For a lab environment, binding to <code>127.0.0.1</code> would be more appropriate unless remote dashboard access is intentional.
Log files in <code>/tmp/</code>	RISK	<code>run.sh</code> lines 46, 51, 56: Background service logs are written to <code>/tmp/noc_*.log</code> with default world-readable permissions.

9. Known Issues / Limitations

9.1 Bugs Found During Inspection

#	FILE	LINE(S)	ISSUE	SEVERITY
1	<code>engine/api.py</code>	36–42	SQLite connection not using context manager (<code>with</code>). If <code>pd.read_sql_query</code> raises, <code>conn.close()</code> is never called → connection leak.	Medium
2	<code>copilot/rag.py</code>	36–40	TF-IDF vector construction has a bug: iterates <code>zip(self.vocab.keys(), vec)</code> where <code>word</code> in the zip is actually a string, but the loop body uses <code>self.vocab[word]</code> for indexing — this works but is inefficient and confusing. The TF multiplication is applied via <code>count</code> (which is <code>vec[idx]</code>), but <code>count</code> is named misleadingly — it's the raw frequency, not a separate variable.	Low
3	<code>run.sh</code>	5	<code>cd "\$SCRIPT_DIR/.."</code> assumes the script is inside a subdirectory. If <code>run.sh</code> is at the project root, this navigates to the parent, potentially breaking <code>PYTHONPATH</code> .	Medium
4	<code>topology.clab.yml</code>	12, 17	Uses <code>frrouting/frr:latest</code> — unpinned container image tag. Builds are non-reproducible.	Low
5	<code>test_config.yml</code>	72–91	Contains real internal network topology IPs (<code>10.50.0.231</code> , <code>192.168.56.1</code>). If published to a public repo, this reveals infrastructure details.	Info

9.2 Architecture / Scalability Concerns

CONCERN	IMPACT
Single-threaded SQLite	SQLite does not support concurrent write access. Under heavy load (multiple simulation daemons or API workers), writes will serialize or fail with <code>database is locked</code> . Not suitable for multi-worker production deployment.
Single XGBoost model for all interfaces	The model is trained only on <code>Branch3-Uplink</code> synthetic data. Predictions for other interfaces use the same model, which may not generalize.
2-second dashboard polling loop	<code>ui/app.py</code> line 416–417 uses <code>time.sleep(2); st.rerun()</code> — this creates continuous re-renders regardless of whether data changed. Resource-intensive on slow hardware.
No model versioning	<code>data/xgboost_model.json</code> is overwritten on re-training with no versioning, rollback capability, or A/B comparison.

CONCERN	IMPACT
Hardcoded relative paths	All file paths (<code>noc-copilot/data/telemetry.db</code> , etc.) are relative to an assumed parent directory. Running from a different CWD will fail silently.
No health check dependencies	<code>/health</code> endpoints return <code>{"status": "ok"}</code> unconditionally without verifying database connectivity, model loading, or RAG readiness.

10. Recommendations / Next Steps

Critical (Fix Immediately)

#	ITEM	FILES AFFECTED
1	Add API authentication — Implement API key or JWT authentication on <code>/predict</code> and <code>/copilot</code> endpoints. At minimum, add a shared secret header check.	<code>engine/api.py</code> , <code>copilot/api.py</code>
2	Bind services to 127.0.0.1 — Change <code>--host 0.0.0.0</code> to <code>--host 127.0.0.1</code> in <code>run.sh</code> unless remote access is explicitly required.	<code>run.sh</code> lines 51, 56, 72

High (Fix Before Deployment)

#	ITEM	FILES AFFECTED
3	Add HTTPS/TLS — Configure Uvicorn with <code>--ssl-keyfile</code> and <code>--ssl-certfile</code> , or deploy behind an HTTPS reverse proxy (nginx/Caddy).	<code>run.sh</code> , new <code>nginx.conf</code>
4	Add rate limiting — Install <code>slowapi</code> or implement custom middleware to prevent API abuse.	<code>engine/api.py</code> , <code>copilot/api.py</code>
5	Sanitize error messages — Replace <code>str(e)</code> in HTTP 500 responses with generic messages. Log detailed errors server-side only.	<code>engine/api.py</code> line 86, <code>copilot/api.py</code> line 51
6	Fix SQLite connection leak — Use <code>with sqlite3.connect(DB_PATH) as conn:</code> context manager in <code>engine/api.py</code> .	<code>engine/api.py</code> lines 36–42
7	Pin dependency versions — Run <code>pip freeze > requirements.lock</code> and reference the lockfile in CI.	<code>requirements.txt</code>
8	Add input validation ranges — Validate <code>risk_score</code> is between 0.0–1.0 and <code>time_to_impact</code> is ≥ -1.0 in the Pydantic model.	<code>copilot/api.py</code> lines 16–20

Medium (Fix for Production Readiness)

#	ITEM	FILES AFFECTED
9	Replace <code>print()</code> with structured logging — Use Python <code>logging</code> module with appropriate levels (DEBUG, INFO, WARNING, ERROR).	All <code>.py</code> files
10	Add Dockerfile — Package the application in a container for reproducible deployment.	New <code>Dockerfile</code>
11	Add security headers — Use FastAPI middleware to add CSP, X-Frame-Options, HSTS, X-Content-Type-Options.	<code>engine/api.py</code> , <code>copilot/api.py</code>
12	Fix health check endpoints — Verify DB connectivity and model status before returning <code>ok</code> .	<code>engine/api.py</code> , <code>copilot/api.py</code>
13	Sanitize LLM prompt inputs — Strip or escape special characters from <code>interface</code> and <code>contributing_signals</code> before injecting into the LLM prompt.	<code>copilot/llm.py</code> lines 55–78
14	Escape RAG content in HTML — Use <code>html.escape()</code> on <code>doc['chunk']</code> and <code>src</code> before passing to <code>unsafe_allow_html=True</code> .	<code>ui/app.py</code> lines 375, 378
15	Remove unused dependencies — Drop <code>scikit-learn</code> , <code>pytest</code> , • <code>pytest-timeout</code> from <code>requirements.txt</code> or actually use them.	<code>requirements.txt</code>
16	Add database migration tooling — Use Alembic or a simple version table to manage schema changes.	<code>faultsim/</code> <code>inject_fault.py</code>

Low (Nice to Have)

#	ITEM	FILES AFFECTED
17	Add graceful signal handling — Catch SIGTERM/SIGINT in <code>inject_fault.py</code> daemon for clean shutdown.	<code>faultsim/inject_fault.py</code> line 84
18	Pin container image tags — Change <code>frrouting/frr:latest</code> to a specific version.	<code>topology.clab.yml</code>
19	Add code coverage reporting — Configure <code>pytest-cov</code> and add coverage thresholds to CI.	New <code>pytest.ini</code> or <code>pyproject.toml</code>
20	Push CI/CD workflow to GitHub — Regenerate token with <code>workflow</code> scope and push <code>.github/workflows/test.yml</code> .	GitHub token settings
21	Move log files from <code>/tmp/</code> — Write logs to <code>data/logs/</code> directory with restricted permissions.	<code>run.sh</code> lines 46, 51, 56
22	Add model versioning — Save timestamped model artifacts and track training metadata.	<code>engine/model.py</code>

End of Report. All findings are based on actual source code inspection. No information was invented or assumed.